Web VAPT Section 09 - Cross-Site Scripting: Reflected and Stored XSS

Professional documentation for XSS validation, browser execution impact, output encoding, and secure remediation.

Enterprise Security Assessment Lab

PROJECT AREA	Web Application Vulnerability Assessment and Penetration Testing
PRIMARY TARGET	DVWA running in local lab environment
TESTING ENVIRONMENT	Kali Linux attack machine with Docker-based vulnerable target
PREPARED BY	Jagriti Banerjee
DOCUMENT TYPE	Individual Web VAPT Attack-Section PDF

AUTHORIZED LAB TESTING ONLY

No live client data. No production systems. Screenshots and outputs must be redacted before public upload.

1. Document Purpose

This individual PDF documents the Web VAPT attack section titled 'Cross-Site Scripting: Reflected and Stored XSS'. It is designed for the Enterprise Security Assessment Lab GitHub portfolio and describes the objective, tooling, commands, sanitized output, evidence mapping, security interpretation, remediation, and redaction requirements for this specific section.

> This document is written as a public portfolio version. It does not contain live client data, production system details, unredacted credentials, session cookies, raw hash dumps, or destructive attack instructions.

Field	Details
Project Area	Web Application Vulnerability Assessment and Penetration Testing
Primary Target	DVWA running in a local lab environment
Testing Environment	Kali Linux attack machine with Docker-based vulnerable target
Testing Style	Reconnaissance -> Enumeration -> Manual validation -> Exploitation proof -> Evidence capture
Methodology Reference	OWASP Web Security Testing methodology, OWASP Top 10, and PTES-style workflow
Prepared By	Jagriti Banerjee

2. Section Overview

Item	Details
Section Number	09
Section Title	Web VAPT Section 09 - Cross-Site Scripting: Reflected and Stored XSS
Risk / Severity Style	High in real-world context
OWASP Mapping	OWASP A03 - Injection
Primary Evidence Count	3

2.1 Objectives

- Validate reflected XSS using controlled lab payloads.
- Validate stored XSS behaviour and source/payload evidence.
- Document browser-side execution risk.
- Explain secure output encoding and CSP-based mitigation.

2.2 Tools Used

Tool	Purpose / Use in this section
Browser	Manual verification and proof screenshots.
Burp Suite	HTTP interception and manual request testing.
DVWA	Intentionally vulnerable application used for validation.

3. Methodology and Execution Workflow

The execution workflow follows a controlled lab approach: confirm authorization and target scope, run only the tools required for the section, save outputs, validate observations manually where applicable, capture screenshots, redact sensitive values, and document findings without exaggeration.

3.1 Command Reference

```
# Harmless lab payload examples
<script>alert('lab')</script>
<script>alert('stored-lab')</script>

# Professional redaction note:
# Do not publish cookies, session identifiers, or payloads designed for theft.
```

3.2 Expected / Sanitized Output

```
Reflected XSS was validated in the DVWA lab.
Stored XSS payload/source evidence was captured.
The payloads were proof-of-execution lab payloads, not credential theft payloads.
```

3.3 Validation Criteria

- The target must be the local DVWA or authorized lab environment only.
- The output must match the intended section objective.
- Evidence screenshots must clearly show the validation result or tool output.
- Any sensitive values must be blurred, cropped, removed, or replaced with placeholders.
- The section must not claim findings that are not supported by evidence.

4. Evidence Mapping

Evidence File	Evidence Type / Reason
15-reflected-xss-alert-popup.png	Reflected XSS validation evidence.
15-reflected-xss-alert-popup-p2.png	Additional reflected XSS evidence.
16-stored-xss-payload-source.png	Stored XSS payload/source evidence.

5. Professional Security Analysis

XSS is an injection class that shifts impact to the user browser. Reflected XSS usually requires victim interaction, while stored XSS persists and can affect multiple viewers.

Stored XSS is often higher impact because it can target administrators or repeated visitors without needing the attacker to send a fresh link each time.

Professional XSS reporting should include affected parameter, context, execution proof, and remediation based on output context.

5.1 Why This Section Matters

This section matters because professional VAPT is not only about running a tool. It requires a clear objective, controlled execution, evidence capture, correct interpretation, and practical remediation. For recruiters and reviewers, this PDF shows that the work was structured, documented, and aligned with real security methodology.

5.2 Common Mistakes to Avoid

- Uploading raw outputs that expose cookies, hashes, paths, or credentials.
- Claiming production-level impact without production context.
- Reporting scanner output as a confirmed vulnerability without validation.
- Using unclear screenshots that do not show the relevant result.
- Mixing unrelated phases in one evidence file without explanation.

6. Risk Analysis

- Session theft if cookies are not protected.
- Phishing and page manipulation.
- Administrator targeting.
- Malicious redirects.
- Account actions through authenticated browser context.

6.1 Real-World Impact Statement

The real-world impact depends on the affected asset, authentication context, data sensitivity, privilege level, exposed functionality, and compensating controls. Because this project uses DVWA in a lab, severity is described as a real-world context estimate rather than a formal client CVSS score.

7. Remediation and Hardening Guidance

- Encode output based on context: HTML, attribute, JavaScript, URL, or CSS.
- Sanitize user-controlled HTML using a safe library.
- Use Content Security Policy as defense-in-depth.
- Set cookies as HttpOnly, Secure, and SameSite.
- Validate input and enforce allowlists where appropriate.

7.1 Verification After Remediation

1. Re-run the original test case in the lab or development environment.
2. Confirm that the previous vulnerable behaviour no longer appears.
3. Check server logs for blocked or rejected attempts.
4. Capture new evidence showing the fix or defensive control.
5. Update the report status from validated/open to fixed/retested if applicable.

8. GitHub Redaction and Publishing Checklist

> Cookies, hashes, credentials, session IDs, shell paths, database output, and private details must be redacted before public upload.

Item	Action Before Upload
PHPSESSID / Cookies	Blur or replace with <REDACTED_COOKIE>.
Password hashes	Blur or replace with <REDACTED_HASH>.
Cracked passwords	Do not publish; replace with <REDACTED_PASSWORD>.
Database dumps	Crop, blur, or summarize instead of uploading raw data.
Webshell paths/files	Do not upload shell source files; redact paths if needed.
Local usernames/paths	Blur if private or unnecessary.
Tool raw outputs	Upload only sanitized outputs or summary documents.

8.1 Safe Git Commands Before Push

```
git status
git diff --cached
git diff --cached | grep -iE "password|secret|token|cookie|PHPSESSID|hash|credential"
```

9. Conclusion

This document completes the individual Web VAPT PDF for Section 09. It documents the section objective, execution workflow, evidence mapping, professional interpretation, risk, remediation, and GitHub safety checks. The content is aligned to the actual Enterprise Security Assessment Lab Web VAPT evidence set and avoids unsupported production claims.